

Homework Exercise Sheet

Ex. 8 Solution

Label (S) : only one correct answer; (M) : more than one correct answer

1. (S) What is the matrix representation of the linear function $\Phi(p) = \int_0^1 p(t)dt, p \in \mathcal{P}_{k-1}$ with respect to the monomial basis $\{1, t, t^2, \dots, t^{k-1}\}$ of $\mathcal{P}_{k-1}$?

Hint: In other words, we are going to find a matrix $\mathbf{M}$ such that for $\mathbf{a} = [\alpha_0, \dots, \alpha_{k-1}]^T \in \mathbb{R}^k$, $b := \mathbf{M}\mathbf{a} \in \mathbb{R}$ is equal to $\int_0^1 \sum_{i=0}^{k-1} \alpha_i t^i dt$.

a) $\begin{bmatrix} 1 & 1 & \cdots & 1 \\ & 1/2 & \cdots & 1/2 \\ & & \ddots & \vdots \\ & & & 1/k \end{bmatrix}$

b) $\begin{bmatrix} 1 & 1/2 & \cdots & 1/k \\ & 1/2 & \cdots & 1/k \\ & & \ddots & \vdots \\ & & & 1/k \end{bmatrix}$

c) $\begin{bmatrix} 1 & 1/2 & \cdots & 1/k \end{bmatrix}$

d) $\begin{bmatrix} 1 \\ 1/2 \\ \vdots \\ 1/k \end{bmatrix}$

Solution:

c). We can expand $\Phi(p)$ into a linear combination of $\{1, t, t^2, \dots, t^{k-1}\}$:

$$\Phi(p) = \int_0^1 \sum_{i=0}^{k-1} \alpha_i t^i dt = \sum_{i=0}^{k-1} \alpha_i \int_0^1 t^i dt = \sum_{i=0}^{k-1} \alpha_i \Phi(t^i) = \mathbf{M}\mathbf{a}.$$

Thus, $\mathbf{M} \in \mathbb{R}^{1 \times k}$, $\mathbf{M}_{0j} = \Phi(t^j) = \int_0^1 t^j dt = \frac{1}{j+1}, 0 \leq j \leq k-1$.

2. (M) Consider the vector space $\mathcal{P}_k = \{t \mapsto \alpha_k t^k + \alpha_{k-1} t^{k-1} + \dots + \alpha_1 t + \alpha_0, \alpha_j \in \mathbb{R}\}$ of polynomials of degree $\leq k$, where $k \in \mathbb{N}$. Additionally, consider $\mathcal{P}_k^{\text{even}} = \{p \in \mathcal{P}_k : p(t) = p(-t), \forall t \in \mathbb{R}\}$ and $\mathcal{P}_k^{\text{odd}} = \{p \in \mathcal{P}_k : p(t) = -p(-t), \forall t \in \mathbb{R}\}$. Choose the correct statements.

a) $\dim \mathcal{P}_4 = 2$

b) $\dim \mathcal{P}_4 = 3$

c) $\dim \mathcal{P}_4 = 4$

d) $\dim \mathcal{P}_4 = 5$

e) $\dim \mathcal{P}_4^{\text{even}} = 2$

f) $\dim \mathcal{P}_4^{\text{even}} = 3$

g) $\dim \mathcal{P}_4^{\text{even}} = 4$

h) $\dim \mathcal{P}_4^{\text{even}} = 5$

i) $\dim \mathcal{P}_4^{\text{odd}} = 2$

j) $\dim \mathcal{P}_4^{\text{odd}} = 3$

k) $\dim \mathcal{P}_4^{\text{odd}} = 4$

l) $\dim \mathcal{P}_4^{\text{odd}} = 5$

Solution:

d), f), i). The dimension of $\mathcal{P}_k$ is equal to the number of coefficients α_j used to uniquely define it. $\mathcal{P}_4^{\text{even}}$ can only be composed of coefficients with even powers of t , that is, α_0, α_2 and α_4 . The number of such coefficients defines its dimension. Similarly, $\mathcal{P}_k^{\text{odd}}$ is composed of coefficients with odd powers of t , that is, α_1 and α_3 .

3. (M) Find the Lagrange polynomials $L_0(x), L_1(x), L_2(x), L_3(x)$ and construct the Lagrange polynomial interpolant $y = p(x)$ for the set of interpolation nodes $\mathbf{x} = [1, 2, 4, 7]$ and $\mathbf{y} = [3, 4, 8, 1]$. Choose the correct statements.
- | | | | |
|--------------------|--------------------|---------------------|--------------------|
| a) $L_0(5) = 1/2$ | b) $L_0(5) = 1/3$ | c) $L_0(5) = 1/4$ | d) $L_0(5) = 1/5$ |
| e) $L_1(5) = -1/2$ | f) $L_1(5) = -3/5$ | g) $L_1(5) = -7/10$ | h) $L_1(5) = -4/5$ |
| i) $L_2(5) = 4/3$ | j) $L_2(5) = -4/3$ | k) $L_2(5) = 3/4$ | l) $L_2(5) = -3/4$ |
| m) $L_3(5) = 1/15$ | n) $L_3(5) = 1/5$ | o) $L_3(5) = 1/3$ | p) $L_3(5) = 2/15$ |
| q) $p(5) = 7.5$ | r) $p(5) = 8.6$ | s) $p(5) = 8.8$ | t) $p(5) = 9.2$ |

Solution:

b), h), i), p), r). Utilize the formula 5.2.2.4 from the lecture document to calculate the Lagrange polynomials. Employ formula 5.2.2.6 to construct the Lagrange polynomial interpolant. $L_0(x) = \frac{1}{18}(-x^3 + 13x^2 - 50x + 56)$, $L_1(x) = \frac{1}{10}(x^3 - 12x^2 + 39x - 28)$, $L_2(x) = \frac{1}{18}(-x^3 + 10x^2 - 23x + 14)$, $L_3(x) = \frac{1}{90}(x^3 - 7x^2 + 14x - 8)$, $p(x) = \frac{1}{15}(-3x^3 + 26x^2 - 42x + 64)$.

4. (M) Select the lines containing errors in the implementation of Vandermonde matrix initialization from interpolation points $\{t_i\}_i$.

```

1 MatrixXd vander(const VectorXd &t) {
2     const VectorXd::Index n = t.size();
3     MatrixXd V(2*n, 2*n);
4     V.col(0) = VectorXd::Ones(n);
5     V.col(n - 1) = t;
6     for (int j = 1; j < n - 2; j++)
7         V.col(j) = (t.array().pow(j)).matrix();
8     return V;
9 }
```

- | | | | |
|-----------|-----------|-----------|-----------|
| a) line 1 | b) line 2 | c) line 3 | d) line 4 |
| e) line 5 | f) line 6 | g) line 7 | h) line 8 |

Solution:

c), e), f). Line 3 should be `MatrixXd V(n,n);`. Line 5 should be `V.col(1) = t;`. Line 6 should be `for (int j = 2; j < n; j++)`.

5. (S) Please indicate the computational complexities of **barycentric interpolation formula**, **Atiken-Neville scheme** and **divided difference** respectively for evaluation of the Lagrange polynomial interpolant at a given point $x_0 \in \mathbb{R}$ provided a given data set $\{t_i, y_i\} \in \mathbb{R}^2, i = 0, \dots, n$.
- | | |
|---|---|
| a) $\mathcal{O}(n), \mathcal{O}(n), \mathcal{O}(n)$. | b) $\mathcal{O}(n^2), \mathcal{O}(n), \mathcal{O}(n)$. |
| c) $\mathcal{O}(n), \mathcal{O}(n^2), \mathcal{O}(n)$. | d) $\mathcal{O}(n), \mathcal{O}(n), \mathcal{O}(n^2)$. |
| e) $\mathcal{O}(n), \mathcal{O}(n^2), \mathcal{O}(n^2)$. | f) $\mathcal{O}(n^2), \mathcal{O}(n), \mathcal{O}(n^2)$. |
| g) $\mathcal{O}(n^2), \mathcal{O}(n^2), \mathcal{O}(n)$. | h) $\mathcal{O}(n^2), \mathcal{O}(n^2), \mathcal{O}(n^2)$. |

Solution:

h). For a single evaluation given a **fixed** data set, none of the three algorithms have any advantage over the plane implementation which is asymptotically $\mathcal{O}(n^2)$ in complexity.

6. (S) Please indicate the computational complexities of **barycentric interpolation formula**, **Atiken-Neville scheme** and **divided difference** respectively for evaluation of the Lagrange polynomial interpolant at multiple points $x_j \in \mathbb{R}, j = 0, \dots, N$ provided a given data set $\{t_i, y_i\} \in \mathbb{R}^2, i = 0, \dots, n$.

- | | |
|--|--|
| a) $\mathcal{O}(n^2N), \mathcal{O}(n^2N), \mathcal{O}(n^2N)$. | b) $\mathcal{O}(n^2 + nN), \mathcal{O}(n^2N), \mathcal{O}(n^2N)$. |
| c) $\mathcal{O}(n^2N), \mathcal{O}(n^2 + nN), \mathcal{O}(n^2N)$. | d) $\mathcal{O}(n^2N), \mathcal{O}(n^2N), \mathcal{O}(n^2 + nN)$. |
| e) $\mathcal{O}(n^2N), \mathcal{O}(n^2 + nN), \mathcal{O}(n^2 + nN)$. | f) $\mathcal{O}(n^2 + nN), \mathcal{O}(n^2N), \mathcal{O}(n^2 + nN)$. |
| g) $\mathcal{O}(n^2 + nN), \mathcal{O}(n^2 + nN), \mathcal{O}(n^2N)$. | h) $\mathcal{O}(n^2 + nN), \mathcal{O}(n^2 + nN), \mathcal{O}(n^2 + nN)$. |

Solution:

f).

- The barycentric interpolation formula precomputes some weights $\{i\}, i = 0, \dots, N$, which costs $\mathcal{O}(n^2)$ while every subsequent evaluation costs $\mathcal{O}(n)$. In total, the asymptotic complexity is $\mathcal{O}(n^2 + nN)$.
- The Atiken-Neville scheme is tailored for a single evaluation with a possibly growing data set. Thus, for each new evaluation point, the algorithm has to be run again from scratch. This implies an asymptotic complexity of $\mathcal{O}(Nn^2)$.
- For the divided difference approach, the coefficients for the New basis are precomputed, which costs $\mathcal{O}(n^2)$. Each subsequent evaluation costs $\mathcal{O}(n)$ **if Horner's scheme is used**. In total, it is $\mathcal{O}(n^2 + nN)$.

7. (S) The following classes are meant to evaluate global polynomial interpolants in different settings:

```

1 class InterpPolyEval_A {
2     public:
3         // Constructor taking the evaluation point x as argument
4         InterpPolyEval_A(double x);
5         // Add another data point (t,v) ∈ ℝ² and update internal information
6         void addPoint(double t, double y);
7         // Return the value of the current interpolating polynomial at the
            evaluation point x
8         double eval(void) const;
9         // ... private data and member functions
10 };

```

```

1 class InterpPolyEval_B {
2     public:
3         // Constructors taking node vector (t₀,...,tₙ) as argument
4         InterpPolyEval_B(const Eigen::VectorXd &t);
5         // Evaluation operator for data (y₀,...,yₙ) computes
6         // p(xₖ) for xₖs passed in x
7         Eigen::VectorXd eval(const Eigen::VectorXd &y,

```

```
8     const Eigen::VectorXd &x) const;
9     // ... private data and member functions
10 };

1 class InterpPolyEval_C {
2     public:
3     // Idle Constructor
4     InterpPolyEval_C();
5     // Add another data point  $(t,y) \in \mathbb{R}^2$  and update internal information
6     void addPoint(double t, double y);
7     // Evaluation of the current interpolating polynomial at many points
8     // passed in x
9     Eigen::VectorXd eval(const Eigen::VectorXd &x) const;
10    // ... private data and member functions
11 };
```

Which of the following algorithms fits the best for the efficient implementation of these three function classes `InterpPolyEval_A`, `InterpPolyEval_B`, `InterpPolyEval_C` respectively?

- a) Barycentric interpolation formula, Atiken-Neville scheme, Divided difference.
- b) Barycentric interpolation formula, Divided difference, Atiken-Neville scheme.
- c) Atiken-Neville scheme, Barycentric interpolation formula, Divided difference.
- d) Atiken-Neville scheme, Divided difference, Barycentric interpolation formula.
- e) Divided difference, Barycentric interpolation formula, Atiken-Neville scheme.
- f) Divided difference, Atiken-Neville scheme, Barycentric interpolation formula.

Solution:

c).

- `InterpPolyEval_A`: Atiken-Neville scheme, see Section 5.2.3.2.
- `InterpPolyEval_B`: Barycentric interpolation formula, see Section 5.2.3.1.
- `InterpPolyEval_C`: Divided difference, see Section 5.2.3.4.

8. (S) What is the interpolated polynomial when doing Hermite interpolation on $f(x) = \frac{1}{\pi} \sin(\pi x)$ at points $x = 0$ and $x = 1$ with matching values at those points up to 1st derivative?

Hint: Check Remark 5.2.2.14 for the definition of Hermite interpolation.

- a) $p(x) = 0$
- b) $p(x) = -x^2 + x$
- c) $p(x) = -\pi x^2 + \pi x$
- d) $p(x) = -\frac{1}{2}x^2 + \frac{1}{2}x$

Solution:

b). It is easy to check for each option if it is the interpolation polynomial by calculating the values and 1st-order derivatives at $x = 0, 1$. All of them match the values at two points, but only b) matches derivatives $p'(0) = f'(0) = 1, p'(1) = f'(1) = -1$. To calculate the result from scratch, one can formulate four equations for polynomial $p(x)$: $p(0) = 0, p(1) = 0, p'(0) = 1, p'(1) = -1$. Therefore, $p(x)$ is at most 3rd order polynomial because they span a 4-dimensional vector space matching the number of equations. Assume $p(x) = a + bx + cx^2 + dx^3$, the equation becomes $a = 0, a + b + c + d = 0, b = 1, b + 2c + 3d = -1$. Solving this linear system gives the solution: $a = 0, b = 1, c = -1, d = 0$, or, $p(x) = -x^2 + x$.

9. **(S)** Given a polynomial $p(x) = 1 + 2x - 3x^2 + 4x^3$, what are the coefficients of the **Newton basis** at coordinates $x = 0, \frac{1}{3}, \frac{2}{3}, 1$?

Hint: we can evaluate the polynomial $p(x)$ at those coordinates and use the divided differences algorithm to get the coefficients.

a) $1, \frac{13}{9}, 1, 4$

b) $1, \frac{31}{9}, 7, 4$

c) $1, \frac{40}{27}, \frac{59}{27}, 4$

d) $1, \frac{13}{9}, 2, 1$

Solution:

a). Evaluating $p(x)$ on the given coordinates results in $1, \frac{40}{27}, \frac{59}{27}, 4$. Then we can go through the divided difference (according to formula 5.2.3.28) on the coordinate-value pairs:

$$\begin{aligned}
 y[x_0] &= 1, y[x_1] = \frac{40}{27}, y[x_2] = \frac{59}{27}, y[x_3] = 4, \\
 y[x_0, x_1] &= \frac{y[x_1] - y[x_0]}{\frac{1}{3} - 0} = \frac{13}{9}, y[x_1, x_2] = \frac{19}{9}, y[x_2, x_3] = \frac{49}{9}, \\
 y[x_0, x_1, x_2] &= \frac{y[x_1, x_2] - y[x_0, x_1]}{\frac{2}{3} - 0} = 1, y[x_1, x_2, x_3] = 5, \\
 y[x_0, x_1, x_2, x_3] &= \frac{y[x_1, x_2, x_3] - y[x_0, x_1, x_2]}{4 - 0} = 4
 \end{aligned}$$